Secure Web Application Firewall: A Lightweight Rule-Based and Rate-Limiting Approach for Web Application Protection

Saidu Sorie Kamara (2300090411) **Sham Wessam Sulaiman** (2300090405) **Dalaa Rafea Hmedan** (2300090408) **Mohammed Alrsheed** (2300090412)

Degree Project Journal Paper
Department of Computer Science / Cybersecurity
2026

Abstract— Web applications are exposed to a wide range of application-layer threats, including SQL injection, cross-site scripting (XSS), automated abuse, and denial-of-service conditions caused by excessive request rates. This paper presents the design of a lightweight Secure Web Application Firewall (WAF) intended to operate as a protective layer between clients and a web application. The proposed WAF combines traffic filtering, rule-based detection of SQL injection and XSS patterns, dynamic rate limiting for denial-of-service mitigation, and comprehensive security logging. The architecture is designed around a reverse-proxy model so that incoming HTTP requests are inspected before reaching the protected application. The project follows an incremental development methodology consisting of traffic filtering, rule-based protection, rate limiting, security testing, and deployment/monitoring. Rather than relying on a single detection technique, the proposed approach combines deterministic security rules with request-rate controls and centralized logging. The study defines an experimental framework using legitimate traffic and controlled attack simulations to evaluate detection effectiveness, false positives, request handling, and mitigation behavior. The resulting design provides a practical foundation for a small-scale WAF suitable for academic experimentation and can be extended with more advanced anomaly detection and machine-learning techniques in future work.

Keywords— Web Application Firewall, WAF, SQL Injection, Cross-Site Scripting, XSS, DDoS Mitigation, Rate Limiting, Reverse Proxy, Web Security, Traffic Filtering.

1. INTRODUCTION

Modern web applications provide essential services through publicly accessible HTTP and HTTPS interfaces. This accessibility also creates an attack surface in which malicious users can submit crafted requests intended to manipulate application behavior, extract data, execute unwanted scripts, or consume excessive resources. Application-layer attacks are particularly important because they may appear similar to legitimate web traffic and can therefore bypass basic network-level controls.

A Web Application Firewall is a security control that analyzes web traffic and applies policies before requests are delivered to an application. A WAF can detect known malicious request patterns, enforce access policies, record security events, and reduce the impact of high-volume request abuse. The effectiveness of a WAF, however, depends on the quality of its rules, its ability to process traffic with acceptable overhead, and its capacity to distinguish malicious requests from legitimate application behavior.

This project proposes a lightweight Secure Web Application Firewall developed as a degree project. The system is organized around five practical capabilities: traffic filtering, rule-based protection against SQL injection and XSS, dynamic rate limiting, comprehensive logging, and controlled security testing. The principal objective is to create a working security layer that can be understood, implemented, tested, and evaluated using a controlled laboratory environment.

2. PROBLEM STATEMENT

Many web applications depend on application-level input handling and server-side controls to prevent malicious requests. When these controls are incomplete, attackers can exploit vulnerable input fields, query parameters, cookies, and HTTP headers. SQL injection can manipulate database queries, while XSS can cause malicious client-side scripts to execute in a victim's browser. In addition, repeated requests from automated clients can exhaust application resources and reduce service availability.

The problem addressed by this project is the absence of a lightweight, integrated protective layer that can inspect incoming web requests, identify common application-layer attack patterns, restrict abusive request rates, and maintain useful security logs. The proposed WAF addresses this problem through a modular reverse-proxy architecture and a set

of explicit security policies.

3. OBJECTIVES

Objective	Description
Traffic Filtering	Inspect incoming HTTP requests and apply configurable allow/deny policies.
Rule-Based Protection	Detect common SQL injection and XSS indicators using security rules.
Dynamic Rate Limiting	Restrict excessive request rates and reduce application-layer denial-of-service impact.
Comprehensive Logging	Record requests, rule matches, rate-limit events, timestamps, and relevant metadata.
Impact Assessment	Evaluate detection, mitigation, false positives, and performance using controlled tests.

4. RELATED WORK

Web application security research has produced several approaches for identifying malicious requests. Signature- and rule-based WAFs remain attractive because their decisions are explainable and can be implemented with relatively modest computational resources. OWASP guidance emphasizes defensive controls such as input validation, output encoding, secure authentication, parameterized queries, and careful handling of untrusted input. A WAF should therefore be treated as an additional defense layer rather than a replacement for secure application development.

Research has also investigated machine learning and anomaly-detection techniques for WAFs. These approaches can potentially identify previously unseen patterns, but they introduce requirements for representative datasets, feature engineering, training, evaluation, and management of false positives. For an undergraduate degree project, a deterministic baseline is useful because individual security rules can be traced from an observed request to an explicit decision.

Rate limiting is another established defensive mechanism. Instead of attempting to classify every high-volume request as malicious, a rate limiter controls the amount of traffic an application must process from a client or other defined key. Combining request inspection with rate limiting allows the proposed WAF to address both content-based attacks and abusive request volume.

5. PROPOSED SYSTEM

The proposed system is a lightweight WAF deployed as a reverse proxy in front of a deliberately vulnerable or test web application. The client sends an HTTP request to the WAF rather than directly to the application. The WAF parses the request, applies filtering and security rules, checks request-rate limits, logs the decision, and either blocks the request or forwards it to the protected application.

5.1 High-Level Architecture

Layer	Main Function
Client	Sends legitimate or malicious HTTP/HTTPS requests.
WAF / Reverse Proxy	Receives requests and coordinates inspection, policy enforcement, rate limiting, logging, and forwarding.
Traffic Filter	Applies allow/deny rules based on request characteristics.
Attack Detection Rules	Checks request components for SQL injection and XSS indicators.
Rate Limiter	Tracks request frequency and temporarily restricts excessive traffic.
Logger	Stores security events and operational information for analysis.
Protected Web Application	Receives only requests permitted by the WAF.

Figure 1. Logical architecture of the proposed Secure WAF.

5.2 Request Processing Workflow

Step 1: The client sends an HTTP request to the WAF. **Step 2:** The WAF extracts relevant request attributes such as method, URI, query parameters, headers, and body data where applicable. **Step 3:** Traffic-filtering policies are evaluated. **Step 4:** SQL injection and XSS detection rules inspect untrusted input for suspicious patterns. **Step 5:** The rate limiter checks the request frequency for the configured client key. **Step 6:** The decision and relevant event metadata are logged. **Step 7:** A request that satisfies the security policy is forwarded; a request that violates a blocking policy is rejected.

6. METHODOLOGY

The project follows an incremental development methodology. Each security capability is implemented and tested before the next capability is integrated. This reduces debugging complexity and allows the security impact of individual modules to be measured.

Phase	Development Activity	Evaluation Focus
1	Traffic filtering tools and test deployment.	Correct allow/deny decisions.
2	Rule-based SQL injection and XSS protection.	Detection and false-positive behavior.
3	Dynamic rate-limiting module and web-server integration.	Request throttling and service availability.
4	Controlled attack simulations and refinement.	Security effectiveness and robustness.
5	Deployment-oriented monitoring and assessment.	Operational suitability and logging quality.

6.1 Rule-Based Detection

The SQL injection module is designed to identify suspicious input patterns associated with SQL manipulation, such as unexpected query operators, comment sequences, tautology-like expressions, and other configurable indicators. The XSS module focuses on potentially executable markup or script-related input, including suspicious script tags, event-handler patterns, and encoded forms that are normalized before evaluation where appropriate.

The design should avoid relying on a single substring match. Normalization, canonicalization, case handling, and careful rule ordering are important because attackers can represent equivalent input in different forms. Rules should also be maintained as configurable policies so that new signatures can be added without redesigning the entire WAF.

6.2 Dynamic Rate Limiting

The rate limiter maintains a request counter for a defined time window and client identifier. When the number of requests exceeds the configured threshold, subsequent requests are delayed or rejected for a defined period. A practical implementation can use a token-bucket or sliding-window strategy. The selected strategy should balance protection with legitimate burst traffic, because an excessively strict threshold can create false positives against normal users.

7. IMPLEMENTATION DESIGN

The proposed implementation can be organized into independent modules so that each security function has a clear responsibility. A lightweight Python/FastAPI-based reverse-proxy implementation is suitable for the academic prototype because it provides an accessible environment for HTTP processing, rule evaluation, logging, and integration with a test web application.

Module	Responsibility
Proxy Controller	Accept requests, coordinate modules, and forward approved requests.
Request Parser	Extract HTTP method, URI, parameters, headers, and body information.
Policy Engine	Apply allow/deny rules and determine the security action.
SQLi Detector	Evaluate normalized input against SQL injection signatures.
XSS Detector	Evaluate normalized input against XSS-related signatures.
Rate Limiter	Track request frequency and enforce thresholds.
Security Logger	Record decisions, alerts, timestamps, and request metadata.
Configuration Layer	Store thresholds and rule settings in a manageable format.

7.1 Security Decision Model

A simple decision hierarchy can be defined as follows: requests that violate an explicit deny policy are blocked; requests matching high-confidence SQL injection or XSS rules are blocked or flagged according to policy; requests exceeding the rate limit are throttled or rejected; all other requests are forwarded. Every security decision should generate a corresponding log event. This creates an auditable relationship between a request, the rule that affected it, and the resulting action.

7.2 Logging and Monitoring

The logging subsystem should capture a timestamp, request method, requested path, client identifier or IP address where appropriate, rule identifier, decision, reason, and rate-limit status. Sensitive data should not be unnecessarily recorded. Logs can later be analyzed to identify repeated attacks, frequently triggered rules, and possible false positives.

8. EXPERIMENTAL TEST PLAN

The evaluation should use a controlled laboratory environment containing the WAF, a test web application, and an attack generation or manual testing component. Tests should be conducted only against systems owned by or explicitly authorized for the project. The objective is to measure the WAF's behavior rather than to attack external systems.

Test Category	Example Test	Expected WAF Behavior
Legitimate Traffic	Normal GET/POST requests and valid application input.	Forward request.
SQL Injection	Controlled malicious SQL-like payloads in test parameters.	Detect and block/flag according to rule.
XSS	Controlled script-related payloads in test input.	Detect and block/flag according to rule.
Rate Abuse	Repeated requests above configured threshold.	Throttle or reject excess requests.
False Positive	Inputs containing legitimate special characters.	Permit legitimate request when no strong rule match exists.
Logging	All test categories.	Create an accurate security event record.

8.1 Evaluation Metrics

The principal evaluation metrics are detection rate, false-positive rate, mitigation effectiveness, processing overhead, and logging completeness. Detection rate can be calculated as the proportion of malicious test requests correctly identified. False-positive rate measures legitimate requests incorrectly blocked. Processing overhead compares request latency with and without the WAF. Rate-limiting effectiveness can be assessed by measuring the proportion of excessive requests prevented from reaching the application.

9. EXPECTED RESULTS AND DISCUSSION

Because the project is being developed as an experimental degree project, results should be reported only after the defined test cases have been executed. The expected outcome is that the WAF will correctly identify a substantial portion of the configured SQL injection and XSS test cases, restrict excessive request rates, and provide traceable security logs. The actual numerical detection and false-positive results should be calculated from the final experiment rather than assumed in advance.

The main engineering trade-off is between security sensitivity and usability. Aggressive rules can increase detection but may also block legitimate input. Conversely, permissive rules can reduce false positives while allowing more malicious traffic to pass. Rate limiting presents a similar trade-off: low thresholds improve resistance to request floods but can affect users whose legitimate behavior produces bursts of requests. Therefore, configuration and empirical testing are central to the proposed design.

The logging component is expected to improve the project's ability to analyze attacks and tune rules. By recording which rule caused a decision, the team can identify rules that are too broad, rules that are frequently triggered, and attack patterns that require additional coverage.

10. SECURITY AND ETHICAL CONSIDERATIONS

All attack simulations must be conducted in an isolated or explicitly authorized environment. The test application should be deliberately vulnerable or configured for safe experimentation, and no unauthorized public systems should be targeted. Security logs should be handled carefully because request metadata may contain sensitive information. The WAF should be presented as a defense-in-depth mechanism and not as a guarantee of complete protection.

11. LIMITATIONS

The proposed prototype is intentionally focused on common application-layer threats and does not represent a complete enterprise WAF. Signature-based rules can miss novel or heavily obfuscated attacks, while poorly designed patterns can generate false positives. Rate limiting based on a simple client identifier can also be bypassed by distributed sources. The project does not claim to replace secure coding, parameterized database queries, output encoding, authentication controls, network protections, or dedicated enterprise DDoS mitigation.

12. FUTURE ENHANCEMENTS

- Integration with a larger and continuously maintained rule set.
- Anomaly detection and machine-learning-assisted classification.
- Distributed rate limiting using shared state across multiple WAF instances.
- Dashboard-based visualization of alerts and traffic trends.
- HTTPS termination and stronger deployment security.
- Containerized deployment with automated CI/CD security testing.
- Threat-intelligence integration and automated rule updates.
- More extensive benchmarking under realistic workloads.

13. CONCLUSION

This paper presented the design of a Secure Web Application Firewall intended to provide a practical defense layer for web applications. The proposed architecture combines reverse-proxy traffic filtering, rule-based SQL injection and XSS detection, dynamic rate limiting, and comprehensive logging. The incremental five-phase development process provides a structured path from basic traffic filtering to controlled security testing and deployment-oriented assessment. The project emphasizes explainable security decisions and measurable evaluation, making it appropriate for an academic prototype. Future work can extend the baseline with anomaly detection, distributed controls, richer monitoring, and automated rule management.

14. REFERENCES

[1] OWASP Foundation, “OWASP Top 10: The Ten Most Critical Web Application Security Risks,” OWASP, current edition.

[2] OWASP Foundation, “Web Application Firewall,” OWASP documentation and community resources.

[3] C. Kruegel and G. Vigna, “Anomaly Detection of Web-based Attacks,” Proceedings of the 10th ACM Conference on Computer and Communications Security, 2003.

[4] V. Paxson et al., “Practical Detection and Prevention of Web Application Attacks,” research literature on application-layer intrusion detection.

[5] M. Roesch, “Snort—Lightweight Intrusion Detection for Networks,” Proceedings of the 13th USENIX Conference on System Administration, 1999.

[6] E. F. K. et al., research literature on machine-learning-based web application firewall and intrusion detection techniques.

[7] NIST, “Security and Privacy Controls for Information Systems and Organizations,” NIST Special Publication 800-53.

[8] NIST, “Guide to Enterprise Patch Management Planning,” NIST Special Publication 800-40.

[9] RFC 6585, “Additional HTTP Status Codes,” Internet Engineering Task Force.

[10] RFC 9110, “HTTP Semantics,” Internet Engineering Task Force, 2022.

APPENDIX A — PROJECT DEVELOPMENT ROADMAP

Phase	Key Deliverables	Status for Journal
Phase 1	Traffic filtering and test deployment.	Planned / prototype stage
Phase 2	SQL injection and XSS rule engine.	Planned / prototype stage
Phase 3	Dynamic rate limiting and integration.	Planned / prototype stage
Phase 4	Controlled attack testing and refinement.	Evaluation stage
Phase 5	Deployment-oriented monitoring and assessment.	Future deployment stage

Note: This journal is written as a formal academic paper based on the current WAF project design. Numerical experimental results have intentionally not been fabricated; they should be inserted after the WAF prototype is tested and measured.